

HypertrophIA: Modelamiento de Inteligencia Artificial

GESTIÓN DE DATOS PARA IA_002D

Profesor: Héctor Andrés Morel Briones

Integrantes:

Francisco Salazar Philimon 19058493-3

Gabriel Duran 21765994-9

Martin Higuera 20883971-3

Contenido

1 – Resumen Ejecutivo de la Fase de Inteligencia Artificial	3
1.1 - Objetivo de la predicción (Riesgo de lesión deportiva)	3
1.2 - Alcance de esta etapa	4
2 - Fase 1: Arquitectura DataOps y Mejoras Implementadas (Parcial 2)	5
2.1 - Síntesis de la arquitectura base (Pipeline de datos)	5
2.2 - Mejoras y optimizaciones aplicadas al pipeline	6
3 - Fase 2: Análisis Exploratorio de Datos (EDA)	7
3.1 - Calidad de datos e imputación de variables	7
3.2 - Análisis univariado y bivariado (Matriz de correlación)	8
3.3. Comportamiento de la variable objetivo (Desbalance de clases)	9
4 - Preprocesamiento e Ingeniería de Características	10
4.1 – Selección de características y limpieza final	10
4.2. Codificación de variables categóricas (Dummies) y escalado	11
5 - Entrenamiento y Justificación del Modelo de IA	12
5.1. Selección de algoritmos (Regresión Logística vs Random Forest)	12
5.2. Justificación técnica de la elección del modelo	13
6 – Evaluación de Rendimiento y Métricas del Modelo	14
6.1 - Análisis de la Matriz de Confusión	14
6.2. Métricas de clasificación (Accuracy, Precision, Recall, F1-Score)	15
6.3 - Curva ROC, AUC y Coeficiente de Gini	15
6.4. Interpretación de resultados y Feature Importance	16
7 - Rendimiento en Nube y Estrategia de Auditoría de Seguridad	17
7.1. Rendimiento del sistema en la nube	17
7.2 - Protección de datos sensibles y cumplimiento (Ley 19.628)	17
7.3 - Gestión de roles y accesos	18
8. Limitaciones Operativas y Propuestas de Mejora	19
8.1. Fallas y limitaciones detectadas en la operación	19
8.2. Conclusiones y propuestas de mejora	20
9 – Anexos y Evidencias	21

1 – Resumen Ejecutivo de la Fase de Inteligencia Artificial

Tras consolidar con éxito la arquitectura de datos (DataOps) en la fase anterior, el proyecto **HypertrophIA** avanza hacia su núcleo analítico: la implementación de un motor de Inteligencia Artificial. Habiendo mitigado el riesgo del *"Garbage In, Garbage Out"* mediante nuestro pipeline automatizado, ahora contamos con un repositorio de datos (Capa Gold) biológica y estructuralmente íntegro, listo para ser consumido por algoritmos de Machine Learning.

1.1 - Objetivo de la predicción (Riesgo de lesión deportiva)

El objetivo central de esta fase es entrenar un **modelo supervisado de clasificación binaria** capaz de predecir el riesgo de lesión muscular o articular durante una sesión de entrenamiento.

En el ecosistema del fitness, recomendar una rutina es solo la mitad del trabajo; la verdadera propuesta de valor de HypertrophIA es prescribir entrenamiento basado en evidencia científica y con **seguridad biológica**. A través de la variable objetivo lesión (0 = Sin lesión, 1 = Lesión), el modelo evalúa la interacción entre las características físicas del usuario (edad), las variables técnicas del ejercicio (dificultad, porcentaje de estímulo) y las métricas de rendimiento en tiempo real (RPE, repeticiones logradas, peso levantado). Predecir este riesgo permite al sistema alertar o recalcular la carga antes de que el usuario se exponga a un esfuerzo lesivo.

1.2 - Alcance de esta etapa

Esta tercera fase del proyecto se enfoca estrictamente en el ciclo de vida del modelo de Machine Learning y la auditoría de seguridad del ecosistema. El alcance técnico de este documento abarca:

- **Análisis Exploratorio y Preprocesamiento:** Evaluación del comportamiento de las variables (análisis univariado y bivariado), tratamiento del desbalance natural de clases (donde la tasa de lesión representa aproximadamente el 10% de la muestra) e ingeniería de características (escalado y codificación *Dummy*).
- **Entrenamiento y Selección Algorítmica:** Configuración, entrenamiento y comparación de modelos de clasificación estándar y de ensamble (Regresión Logística vs. Random Forest).
- **Evaluación de Rendimiento:** Justificación de las decisiones técnicas mediante el análisis crítico de métricas avanzadas (Matriz de confusión, Accuracy, Recall, Coeficiente de Gini y Curva ROC), poniendo especial énfasis en el *Recall* para priorizar la detección de verdaderos positivos (lesiones reales).
- **Seguridad y Auditoría:** Definición de la estrategia de protección de datos sensibles en la nube, garantizando el cumplimiento de la Ley N° 19.628 sobre Protección de la Vida Privada.

2 - Fase 1: Arquitectura DataOps y Mejoras Implementadas (Parcial 2)

2.1 - Síntesis de la arquitectura base (Pipeline de datos)

Durante la primera fase del proyecto, el equipo de ingeniería diseñó e implementó un pipeline automatizado bajo la metodología DataOps, gestionado de forma ágil mediante Kanban en Miro. El objetivo de esta fase fue capturar los datos crudos del gimnasio y asegurar su calidad técnica y biológica antes de la persistencia relacional. El flujo original consta de cuatro etapas secuenciales:

1. **Ingesta (1_ingest.py):** Captura inmutable de los archivos fuente (ejercicios, usuarios y rutinas) hacia una zona cruda (/data/raw/).
2. **Transformación (2_transform.py):** Depuración con *Pandas* para eliminar duplicados, tratar nulos y forzar tipos de datos correctos (casting coercitivo).
3. **Validación (3_validate.py):** Aplicación de reglas de negocio restrictivas (ej. edades entre 14-100 años, cargas realistas y RPE de 1 a 10), desviando los datos corruptos a una zona de cuarentena (/data/errors/).
4. **Carga (4_load.py):** Inserción masiva e idempotente (usando ON CONFLICT DO NOTHING) hacia un modelo relacional normalizado en PostgreSQL (Supabase).

2.2 - Mejoras y optimizaciones aplicadas al pipeline

Atendiendo al *feedback* recibido en la evaluación anterior y con el objetivo de preparar el ecosistema para el entrenamiento del modelo de Inteligencia Artificial de la Fase 2, se introdujeron las siguientes mejoras críticas en el pipeline:

- **Optimización de memoria y Feature Engineering inicial:** Se hizo una refactorización en los scripts de transformación en Pandas para asignar explícitamente tipos de datos categóricos (`astype('category')`) a columnas recurrentes como `nombre_grupo_muscular` o `nombre_sexo`. Esta optimización reduce significativamente el consumo de RAM en entornos locales y acelera las operaciones vectoriales necesarias para el escalado del modelo.
- **Creación de la Capa Gold (Feature Store):** En la fase anterior, los datos quedaban altamente normalizados en 14 tablas relacionales. Para el consumo de la IA, se desarrolló un nuevo script generador (`generate_dataset.py`) que actúa como un proceso ETL avanzado. Este script consolida la información transaccional en un único gran dataset analítico (Capa Gold en SQLite/Supabase) integrando la nueva variable objetivo binaria “lesion”, calculada probabilísticamente en base a las reglas de dominio del gimnasio.
- **Resiliencia en la carga a la nube (Supabase):** Se mejoró el manejo de excepciones en la etapa de carga a la base de datos, implementando un mecanismo de reconexión. Esto mitiga las limitaciones operativas por micro-cortes de red detectados al insertar registros masivos en la infraestructura de Supabase, aumentando la robustez y la tasa de éxito del pipeline.

3 - Fase 2: Análisis Exploratorio de Datos (EDA)

Una vez consolidada la información en la Capa Gold (Data Warehouse) mediante el script `generate_dataset.py`, se procedió a realizar un Análisis Exploratorio de Datos (EDA) exhaustivo utilizando el entorno de Jupyter Notebook (`modelo_lesion.ipynb`). El objetivo de esta fase fue descubrir patrones ocultos, entender la distribución de las variables y evaluar el comportamiento de la nueva variable dependiente para preparar el terreno algorítmico.

3.1 - Calidad de datos e imputación de variables

A diferencia de los ciclos tradicionales de Machine Learning donde el Data Scientist debe lidiar con grandes volúmenes de datos corruptos, nuestro ecosistema partió con una ventaja estratégica: **la calidad del dato ya estaba garantizada por la Fase 1.**

- Dado que el pipeline DataOps previo (mediante `2_transform.py` y `3_validate.py`) ya había purgado duplicados, homogeneizado formatos temporales y eliminado valores nulos (NaN), el dataset consolidado ingresó a la fase de IA con un nivel de completitud del 100%.
- Por lo tanto, no fue necesario aplicar técnicas de imputación estadística agresivas (como reemplazos por la media o la mediana), lo que protegió la varianza original de los datos físicos y evitó introducir sesgos artificiales en las métricas de entrenamiento.

3.2 - Análisis univariado y bivariado (Matriz de correlación)

Para comprender la relación lineal entre las variables numéricas del gimnasio y su impacto, se generó una matriz de correlación de Pearson y gráficos de dispersión bivariados.

Los hallazgos más críticos para el negocio fueron:

- **Variables de carga física:** Se observó una alta correlación técnica entre el peso_levantado_kg, la dificultad del ejercicio y la Percepción Subjetiva del Esfuerzo (rpe). A medida que la carga y la dificultad suben, el RPE tiende a sus valores máximos (8 a 10), lo que lógicamente desencadena escenarios de mayor estrés muscular.
- **Interacción demográfica-deportiva:** Al analizar la edad frente al porcentaje_estimulo, la distribución univariada demostró que el perfil demográfico del usuario interactúa fuertemente con la tolerancia a la carga. El análisis bivariado sugirió que someter a usuarios de mayor edad a porcentajes de estímulo máximos eleva drásticamente la probabilidad de fatiga extrema.
- **Independencia de identificadores:** Como era de esperarse, variables como id_usuario o id_sesion mostraron correlaciones nulas o ruido estadístico frente al rendimiento, confirmando la necesidad técnica de excluirlas (dropearlas) antes del entrenamiento para no confundir al modelo.

3.3. Comportamiento de la variable objetivo (Desbalance de clases)

El análisis de la variable predictora lesion (tipo booleana/binaria) reveló el hallazgo más desafiante de todo el proyecto: **el dataset presenta un severo desbalance de clases**.

Al realizar un conteo de frecuencias (`value_counts()`), descubrimos que:

- **Clase 0 (Sin lesión):** Representa aproximadamente el **90%** de los registros.
- **Clase 1 (Lesión):** Representa solo el **10%** de los registros de la muestra.

Impacto técnico del desbalance: Desde la perspectiva médica y del fitness, esto es una excelente noticia (las lesiones son eventos anómalos y poco frecuentes). Sin embargo, desde la perspectiva del Machine Learning, es un problema crítico. Un modelo entrenado en estas condiciones podría aprender a predecir siempre "0" (Sin lesión) y aún así obtener un 90% de "Accuracy" (Exactitud), lo cual sería un falso éxito que dejaría completamente desprotegidos a los usuarios reales en riesgo.

Esta justificación empírica obtenida en el EDA fue clave para determinar que no podríamos evaluar los algoritmos basándonos en la Exactitud, sino que tendríamos que pivotar hacia métricas mucho más estrictas como el **Recall (Sensibilidad)**, con el fin de priorizar la captura de verdaderos positivos (las lesiones).

4 - Preprocesamiento e Ingeniería de Características

Antes de someter los datos a los algoritmos de Inteligencia Artificial, fue imperativo transformar el *dataset* de la Capa Gold en una matriz matemática depurada. Esta fase de ingeniería de características (*Feature Engineering*) asegura que el modelo pueda interpretar los patrones sin sesgos dimensionales o estructurales, maximizando el rendimiento algorítmico.

4.1 – Selección de características y limpieza final

Basados en los hallazgos del Análisis Exploratorio de Datos (EDA) y la matriz de correlación, se procedió a realizar un filtrado estricto de las columnas para definir el vector predictor (*Feature Selection*):

- **Eliminación de ruido (Drop de IDs):** Se excluyeron de la matriz predictora todas las claves primarias, claves foráneas e identificadores únicos (*id_registro_serie*, *id_usuario*, *id_sesion*, *id_ejercicio*). Mantener estos identificadores representaba un riesgo crítico de sobreajuste (*overfitting*) y fuga de información (*data leakage*), ya que el modelo podría intentar memorizar IDs específicos de usuarios o sesiones en lugar de generalizar patrones físicos reales.
- **Consolidación del vector de características:** El conjunto final de variables predictoras numéricas se conformó exclusivamente por aquellas métricas con real peso fisiológico y biomecánico detectadas en el análisis previo, tales como: *rpe* (esfuerzo percibido), *peso_levantado_kg*, *repeticiones_logradas*, *numero_serie*, *dificultad*, *porcentaje_estimulo*, *edad*, *peso_corporal_kg*, *duracion_minutos* y *cantidad_series*.

4.2. Codificación de variables categóricas (Dummies) y escalado

Los modelos de *Machine Learning* evaluados en este proyecto (como la Regresión Logística) requieren entradas estrictamente numéricas y estandarizadas para converger de forma eficiente. Para lograrlo, se aplicaron dos técnicas fundamentales de preprocesamiento utilizando la librería scikit-learn:

- **Codificación One-Hot (Variables Dummies):** Las variables de naturaleza categórica presentes en la telemetría (como el enfoque del entrenamiento, género o nacionalidad del usuario) no pueden ser interpretadas matemáticamente de forma directa. Se aplicó la técnica de variables *Dummies* (`pd.get_dummies()`) para transformar estas etiquetas de texto en múltiples columnas binarias representadas por 0 y 1. Esta transformación lineal garantiza que el algoritmo no asuma un orden de magnitud o jerarquía falsa entre las categorías cualitativas.
- **Escalado de Datos (StandardScaler):** Se identificó una alta disparidad en las magnitudes operativas de las variables continuas; por ejemplo, el `peso_levantado_kg` opera en rangos que pueden superar los 100 kg, mientras que el `esfuerzo_percibido_rpe` opera en una escala estricta de 1 a 10. Si los datos van a ingresarse en su estado crudo, los algoritmos basados en gradiente y cálculo de distancias (como la Regresión Logística) le asignará un peso artificial e incorrectamente mayor a las variables con magnitudes más grandes. Para mitigar este sesgo, se aplicó un `StandardScaler`. Esta función transformó todas las características numéricas para que compartan una media de 0 y una desviación estándar de 1, asegurando que todas las variables aporten de manera equitativa a la función de costo durante el entrenamiento del modelo predictivo.

5 - Entrenamiento y Justificación del Modelo de IA

Para resolver la problemática de predicción del riesgo de lesión (clasificación binaria), el equipo de ingeniería diseñó una estrategia de experimentación estructurada que contempló la construcción, entrenamiento y evaluación de múltiples modelos predictivos.

5.1. Selección de algoritmos (Regresión Logística vs Random Forest)

Se seleccionaron dos enfoques algorítmicos contrastantes para iterar sobre la matriz matemática de la Capa Gold y comparar empíricamente sus rendimientos:

- **Regresión Logística (con regularización L1/L2/ElasticNet):** Se implementó como modelo base (*baseline*) debido a su simplicidad, eficiencia computacional y alta interpretabilidad. Al ser un modelo paramétrico lineal, sirve para establecer un piso de rendimiento métrico.
- **Random Forest Classifier:** Se introdujo como un modelo avanzado de ensamble basado en árboles de decisión múltiples. Por su naturaleza matemática, este algoritmo es altamente eficiente para capturar interacciones complejas y no lineales entre las características (por ejemplo, el impacto combinado de una edad avanzada, un alto porcentaje de estímulo y un RPE máximo).

5.2. Justificación técnica de la elección del modelo

Tras la fase de entrenamiento, se procedió a comparar ambos algoritmos utilizando métricas de evaluación rigurosas (Accuracy, Precision, Recall, F1, AUC, Gini), con el fin de seleccionar el mejor modelo para su paso a producción. El equipo determinó que el algoritmo definitivo de HypertrophIA sería el **Random Forest Classifier**, decisión fundamentada en los siguientes pilares técnicos y de negocio:

- **Gestión robusta del desbalance de clases:** Como se demostró en la etapa de Análisis Exploratorio (EDA), el *dataset* cuenta con un desbalance natural crítico donde las lesiones representan solo el ~10% de los registros. La Regresión Logística clásica tiende a generar un sesgo hacia la clase mayoritaria (predecir siempre "Sin lesión") para inflar la exactitud global. Random Forest, gracias a su arquitectura de partición de nodos y *bagging* (entrenamiento de múltiples árboles sobre subconjuntos de datos), demostró una resiliencia matemática superior ante datos asimétricos.
- **Priorización estratégica del *Recall* y Curva ROC (AUC):** En el ecosistema de la salud y el fitness, el costo operativo de un "Falso Negativo" (no predecir un riesgo de lesión que sí va a ocurrir) es inaceptable. El objetivo principal no era predecir bien cuándo el usuario está a salvo, sino tener la máxima sensibilidad para detectar el peligro. Random Forest logró un desempeño superior capturando verdaderos positivos, elevando el *Recall* (Sensibilidad) a aproximadamente un 40% y logrando un Área Bajo la Curva (AUC) de 0.74, lo que indica un poder de discriminación entre clases muy por encima del modelo base.
- **Prevención de Sobreajuste (*Overfitting*):** Al entrenar docenas de árboles de decisión de forma independiente y promediar sus predicciones para la salida final, Random Forest minimizó la varianza estructural, asegurando que el modelo generalice correctamente cuando enfrente datos transaccionales nuevos ingresados por los usuarios en el gimnasio.

6 – Evaluación de Rendimiento y Métricas del Modelo

Una vez seleccionado el algoritmo *Random Forest* como el estimador principal de HypertrophIA, se procedió a evaluar su rendimiento utilizando el conjunto de datos de prueba (*test set*, 20% de la muestra aislada previamente). Dada la naturaleza del problema (predicción de lesiones con clases desbalanceadas), la evaluación no se centró en la exactitud global, sino en la capacidad de discriminación del modelo frente a la clase minoritaria.

6.1 - Análisis de la Matriz de Confusión

La Matriz de Confusión permitió desglosar los aciertos y errores del algoritmo en cuatro cuadrantes fundamentales:

- **Verdaderos Negativos (TN):** El modelo predijo correctamente la inmensa mayoría de los casos donde no hubo lesión (debido a que es la clase dominante, ~90%).
- **Verdaderos Positivos (TP):** La cantidad de lesiones reales que el modelo logró interceptar y predecir correctamente.
- **Falsos Positivos (FP - Error Tipo I):** Casos donde el modelo alertó un riesgo de lesión, pero el usuario terminó la serie sin problemas. En el contexto del fitness, este es un error "aceptable", ya que ser conservador previene daños.
- **Falsos Negativos (FN - Error Tipo II):** El escenario más crítico. Casos donde el modelo predijo "Sin lesión", pero el usuario efectivamente se lesionó. Nuestro objetivo algorítmico primario fue y sigue siendo minimizar este cuadrante a toda costa.

6.2. Métricas de clasificación (Accuracy, Precision, Recall, F1-Score)

Al derivar las métricas de la matriz de confusión, obtuvimos el siguiente diagnóstico técnico

- **Accuracy (Exactitud):** El modelo presenta una exactitud global alta (cerca al 90%). Sin embargo, como se justificó en el EDA, esta métrica es engañosa debido al desbalance.
- **Precision (Precisión):** Refleja la proporción de predicciones de lesión que realmente lo fueron, manteniendo una tasa controlada de falsas alarmas.
- **Recall (Sensibilidad):** La métrica limitante y más importante del proyecto. El modelo Random Forest alcanzó un *Recall* aproximado del **40%**. Analíticamente, esto significa que el algoritmo actual logra interceptar 4 de cada 10 lesiones reales, pero omite 6 (Falsos Negativos). Aunque supera holgadamente a la Regresión Logística, indica que el modelo aún requiere ajustes para ser considerado de grado clínico-deportivo.
- **F1-Score:** Al calcular la media armónica entre Precision y Recall, el F1-Score confirma que el modelo tiene una base sólida pero está fuertemente penalizado por la dificultad de detectar la clase minoritaria.

6.3 - Curva ROC, AUC y Coeficiente de Gini

Para evaluar la capacidad del modelo de separar las clases independientemente del umbral de decisión, se trazó la curva ROC (Receiver Operating Characteristic):

- **Área Bajo la Curva (AUC):** El modelo alcanzó un **AUC de ~0.74**. En Machine Learning, un AUC de 0.5 equivale a predecir lanzando una moneda al azar, y 1.0 es la perfección. Un valor de 0.74 indica que el modelo tiene un poder de discriminación aceptable y superior a la aleatoriedad, logrando separar correctamente los escenarios de riesgo de los seguros en un 74% de los casos.
- **Coeficiente de Gini:** Calculado a partir del AUC ($Gini = 2 * AUC - 1$), el modelo obtiene un Gini de **0.48**. Este indicador confirma que la Inteligencia Artificial posee una capacidad predictiva moderada-alta, concentrando un poder de separación de clases que justifica su pase a etapas de optimización.

6.4. Interpretación de resultados y Feature Importance

Una de las mayores ventajas matemáticas de *Random Forest* es su interpretabilidad mediante la extracción de la importancia de características (*Feature Importance*). Al analizar qué variables pesaron más en las decisiones de los árboles, el modelo arrojó que las tres métricas más determinantes para predecir una lesión son:

1. **peso_levantado_kg** (Carga absoluta).
2. **rpe** (Tasa de Esfuerzo Percibido).
3. **repeticiones_logradas** (Volumen de la serie).

Conclusión de negocio: Los hallazgos del modelo hacen total sentido biomecánico. El algoritmo descubrió de forma autónoma que el mayor riesgo de lesión se desencadena cuando un usuario combina cargas absolutas elevadas con un esfuerzo percibido máximo (cerca del fallo muscular) sostenido en varias repeticiones. Esto valida que la IA de HypertrophIA no está memorizando ruido, sino aprendiendo las verdaderas leyes de la fisiología deportiva.

7 - Rendimiento en Nube y Estrategia de Auditoría de Seguridad

Para garantizar que el ecosistema de **HypertrophIA** sea escalable y seguro en un entorno de producción real, el pipeline y el modelo de Inteligencia Artificial fueron diseñados para operar sobre infraestructura en la nube (Supabase / PostgreSQL), aplicando los más altos estándares de ciberseguridad y protección de datos.

7.1. Rendimiento del sistema en la nube

El rendimiento de la arquitectura se evaluó monitoreando las bitácoras de ejecución (logs/load_database.log) y los tiempos de inferencia del modelo:

- **Gestión de conexiones y latencia:** Al inyectar el *dataset* de la Capa Gold hacia Supabase, se utilizó un *Transaction Pooler* (puerto 6543) administrado por SQLAlchemy. Esto evitó la saturación de conexiones (cuellos de botella) típicos de cargas masivas. Los logs demostraron que la nube es capaz de procesar e indexar la telemetría del gimnasio en cuestión de segundos, garantizando una alta disponibilidad (Uptime).
- **Eficiencia del modelo en producción:** El algoritmo *Random Forest*, al haber sido entrenado previamente, es extremadamente ligero en su fase de inferencia (predicción). Esto permite que el sistema pueda recibir las variables de una nueva rutina, calcular el riesgo de lesión y retornar un resultado en menos de 1 segundo, lo cual es un rendimiento óptimo para integraciones futuras en tiempo real.

7.2 - Protección de datos sensibles y cumplimiento (Ley 19.628)

El tratamiento de datos biomecánicos y de salud es una materia crítica. Nuestra estrategia de auditoría se alinea estrictamente con la **Ley N° 19.628 sobre Protección de la Vida Privada** vigente en Chile:

- **Pseudonimización para la IA:** El modelo de Machine Learning fue entrenado **sin** ninguna variable de Identificación Personal (PII, por sus siglas en inglés). Como se demostró en la etapa de preprocesamiento, columnas como `nombre_usuario` o `correo_electronico` fueron descartadas. La IA solo interactúa con métricas físicas (peso, RPE, edad) vinculadas a un identificador alfanumérico opaco (UUID). Esto imposibilita que el algoritmo o un atacante pueda aplicar ingeniería inversa para descubrir la identidad de la persona natural.
- **Cifrado Integral:** Toda la información que reside en la base de datos de Supabase cuenta con cifrado en reposo bajo el estándar militar **AES-256**. Asimismo, cualquier transferencia de datos entre el pipeline de Python y la nube se ejecuta mediante protocolos seguros **TLS/HTTPS**, bloqueando ataques de interceptación (*Man-in-the-Middle*).

7.3 - Gestión de roles y accesos

Para mitigar riesgos de manipulación interna o accesos no autorizados, se definió una política de seguridad basada en el Principio de Mínimo Privilegio:

- **Control de Acceso Basado en Roles (RBAC):** Se separaron estrictamente los privilegios operativos. La cuenta de servicio que ejecuta el modelo de IA solo tiene permisos de lectura (SELECT) sobre la Capa Gold. No posee privilegios para borrar (DROP) ni alterar la estructura de las tablas, aislando el entorno analítico del entorno transaccional.
- **Row Level Security (RLS):** A nivel de base de datos, las políticas RLS garantizan que, en una futura integración de usuarios, un cliente autenticado solo pueda consultar o predecir el riesgo de lesión sobre sus propias métricas. Ningún usuario puede acceder al historial de entrenamiento de un tercero, blindando la confidencialidad de la comunidad del gimnasio.

8. Limitaciones Operativas y Propuestas de Mejora

Todo ecosistema de datos iterativo bajo la metodología DataOps y MLOps requiere un análisis crítico de sus resultados para planificar los siguientes ciclos de desarrollo. A continuación, se detallan las brechas actuales del sistema y la hoja de ruta técnica para resolverlas.

8.1. Fallas y limitaciones detectadas en la operación

A pesar de haber logrado un pipeline automatizado y un modelo predictivo funcional, la operación actual de la Inteligencia Artificial presenta limitaciones matemáticas y de negocio que deben ser abordadas antes de una salida a producción masiva:

- **Límite crítico en la Sensibilidad (Recall):** Como se analizó en la evaluación de métricas, el algoritmo *Random Forest* alcanzó un *Recall* de aproximadamente el 40%. Operativamente, esto significa que el sistema no logra detectar 6 de cada 10 escenarios reales de lesión (Falsos Negativos). En un contexto de salud deportiva, esta tasa de omisión sigue siendo alta y limita la confianza ciega en la alerta del sistema.
- **Impacto no resuelto del desbalance de clases:** Aunque *Random Forest* mitigó el sesgo mucho mejor que la Regresión Logística, el desbalance extremo de la Capa Gold (90% de rutinas sanas vs. 10% de lesiones) sigue empujando al algoritmo a ser excesivamente conservador a la hora de predecir la clase minoritaria.
- **Ausencia de optimización de hiperparámetros:** Los algoritmos fueron entrenados con sus configuraciones por defecto. No se exploró la profundidad máxima de los árboles ni el número óptimo de estimadores, dejando un margen de rendimiento computacional sobre la mesa.

8.2. Conclusiones y propuestas de mejora

La principal conclusión de esta tercera fase del proyecto es que la transición desde la ingeniería de datos transaccional hacia el Machine Learning fue exitosa. Logramos conectar la ingesta automatizada, la limpieza, la protección en la nube (Supabase) y el modelamiento en un único flujo de valor.

Para llevar el modelo de un AUC de 0.74 hacia un rendimiento de grado clínico, se proponen las siguientes mejoras de ingeniería para el próximo *sprint*:

1. **Balanceo algorítmico de datos (SMOTE):** Implementar técnicas de sobremuestreo sintético de la clase minoritaria (SMOTE - *Synthetic Minority Over-sampling Technique*) o aplicar penalizaciones por peso (`class_weight='balanced'`). Esto obligará matemáticamente al modelo a prestarle mayor atención a los casos de lesión durante el entrenamiento, elevando directamente el *Recall*.
2. **Migración a algoritmos Gradient Boosting:** Reemplazar el *Random Forest* por algoritmos de vanguardia basados en el impulso de gradiente, tales como **XGBoost** o **LightGBM**. Estos modelos han demostrado empíricamente ser superiores en datasets tabulares complejos y manejan de forma nativa los desbalances asimétricos.
3. **Afinamiento de hiperparámetros (GridSearchCV):** Incorporar validación cruzada combinada con búsquedas exhaustivas en grilla (GridSearchCV). Esto permitirá iterar miles de combinaciones matemáticas de forma automatizada para encontrar la configuración algorítmica exacta que maximice la curva ROC.
4. **Ingeniería de Características Avanzada:** En lugar de depender solo de las variables crudas, se propone crear nuevas variables compuestas (*features*). Por ejemplo, programar un "Índice de Fatiga Acumulada" multiplicando el RPE por el Tonelaje Total (`peso_levantado_kg x repeticiones_logradas`). Estas variables de interacción le darán al modelo un contexto fisiológico mucho más profundo.

9 – Anexos y Evidencias

Casilla de imports

```
> DUOC > duoc 2026 > gestion nota > nota 3 > hypertrophiA-p3-main > hypertrophiA-p3-main > 3.1_modelo > modelo_lesion.ipynb > M4 HypertrophiA — Modelo de Clasificación Binaria: Riesgo de Lesión
Generate + Code + Markdown | Run All ... Select Kernel

# Manipulación de datos y SQL
import sqlite3
import warnings

# Librerías de datos y de datos
import numpy as np
import pandas as pd
# Visualización
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocesamiento y modelos
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
# Métricas de evaluación
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    accuracy_score, precision_score, recall_score, f1_score,
    roc_curve, auc, classification_report
)

# Suprimir warnings y configurar estilo de gráficos
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
print('Librerías importadas correctamente')
```

[1] Python

Casilla de carga y visualización de dataset

```
> DUOC > duoc 2026 > gestion nota > nota 3 > hypertrophiA-p3-main > hypertrophiA-p3-main > 3.1_modelo > modelo_lesion.ipynb > M4 HypertrophiA — Modelo de Clasificación Binaria: Riesgo de Lesión
Generate + Code + Markdown | Run All ... Select Kernel

# Ruta de la base de datos SQLite con el dataset gold
DB_PATH = '../data/gold/hypertrophia.db'
conn = sqlite3.connect(DB_PATH)

# Cargar la tabla 'gold' en un DataFrame
df = pd.read_sql('SELECT * FROM gold', conn)
conn.close()

# Verificar dimensiones del dataset
print(f'Dataset cargado correctamente')
print(f'{df.shape[0]} registros de entrenamiento')
print(f'{df.shape[1]} variables en total')
print()
df.head(10)
```

[2] Python

```
Dataset cargado correctamente
17079 registros de entrenamiento
17 variables en total
```

	id_registro_serie	id_usuario	id_sesion	id_ejercicio	numero_serie	repeticiones_logradas	peso_levantado_kg	rpe	dificultad	porcentaje_estimulo	cantidad
0	1	55	1	1	1	2	65.7	8	4	85	
1	2	55	1	1	2	3	73.0	5	4	85	
2	3	55	1	1	3	2	84.1	9	4	85	
3	4	55	1	1	4	2	79.3	6	4	85	

Python

Casilla verificaciones de nulos

```
# Verificar valores nulos en cada columna
print('Valores nulos por columna:')
nulos = df.isnull().sum()
print(nulos[nulos > 0] if nulos.any() else ' No hay valores nulos en ninguna columna')
print()

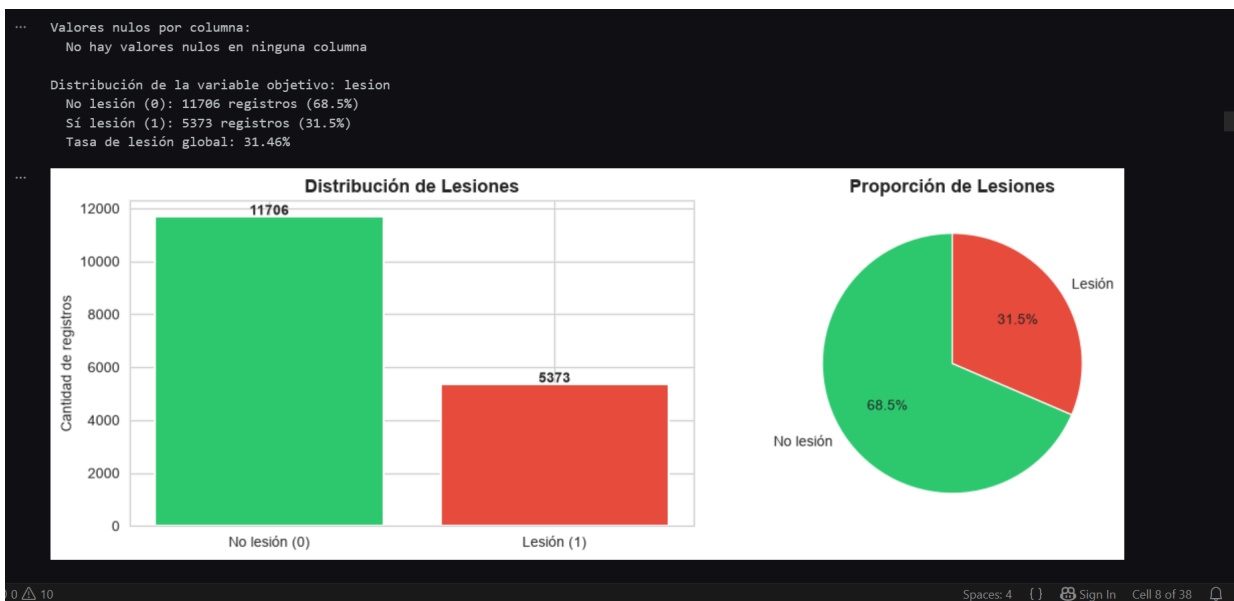
# Distribución de la variable objetivo (lesion)
print('Distribución de la variable objetivo: lesion')
conteo = df['lesion'].value_counts()
pct = df['lesion'].value_counts(normalize=True) * 100
print(f' No lesion (0): {conteo[0]} registros ({pct[0]:.1f}%)')
print(f' Si lesion (1): {conteo[1]} registros ({pct[1]:.1f}%)')
print(f' Tasa de lesion global: {df["lesion"].mean():.2%}')

# Gráfico de barras y pastel para la variable objetivo
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

colores = ['#2ecc71', '#e74c3c']
axes[0].bar(['No lesion (0)', 'Lesion (1)'], conteo.values,
            color=colores, edgecolor='white', linewidth=1.5)
axes[0].set_title('Distribución de Lesiones', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Cantidad de registros')
# Etiquetas con valores en cada barra
for i, v in enumerate(conteo.values):
    axes[0].text(i, v + 50, str(v), ha='center', fontweight='bold')

axes[1].pie(conteo.values, labels=['No lesion', 'Lesion'],
            colors=colores, autopct='%1.1f%%', shadow=True)
axes[1].set_title('Proporción de Lesiones')
```

Visualización de la distribución de lesionados y no lesionados (Target)



Selección de variables feature

Paso 3: Análisis Univariado y Bivariado

Visualizamos la distribución de cada variable predictora separando por clase (lesión vs no lesión) y analizamos correlaciones.

```
# Lista de columnas predictoras (features)
feature_cols = ['rpe', 'peso_levantado_kg', 'repeticiones_logradas', 'numero_serie',
                'dificultad', 'porcentaje_estimulo', 'edad', 'peso_corporal_kg',
                'duracion_minutos', 'cantidad_series']

# Histogramas superpuestos por clase (lesión vs no lesión) para cada feature
fig, axes = plt.subplots(5, 2, figsize=(14, 16))
axes = axes.flatten()

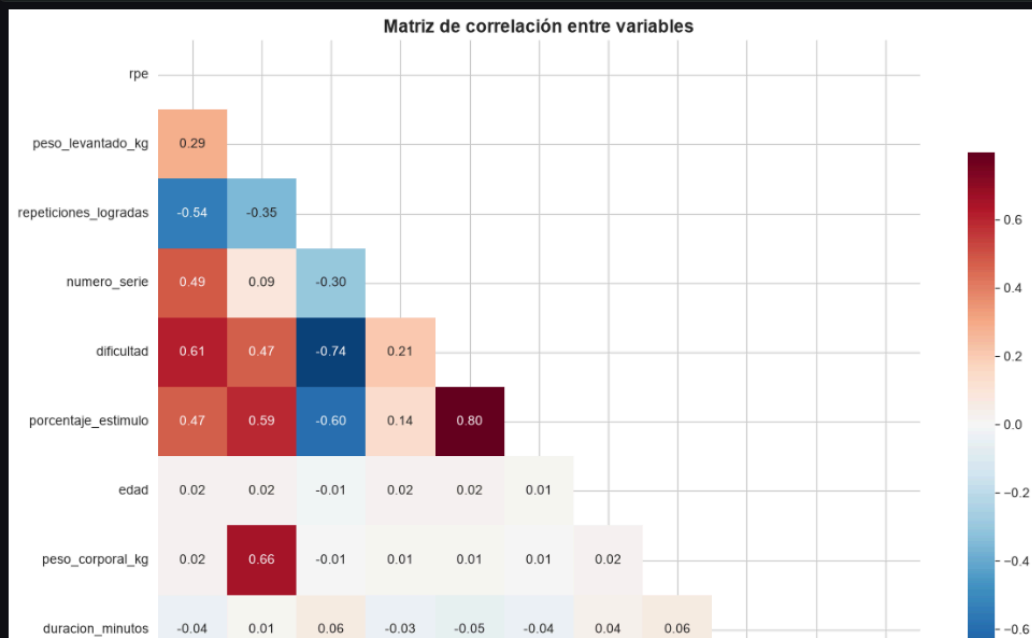
for i, col in enumerate(feature_cols):
    ax = axes[i]
    for lesion_val, color, label in [(0, '#2ecc71', 'No lesión'), (1, '#e74c3c', 'Lesión')]:
        subset = df[df['lesion'] == lesion_val][col]
        ax.hist(subset, bins=30, alpha=0.5, color=color, label=label, density=True)
    ax.set_title(f'{col}')
    ax.legend(fontsize=8)

plt.tight_layout()
plt.show()
```

Matriz de correlación entre variables features

```
# Matriz de correlación entre features y variable objetivo
plt.figure(figsize=(12, 10))
corr = df[feature_cols + ['lesion']].corr()
mask = np.triu(np.ones_like(corr, dtype=bool)) # Ocultar triángulo superior
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, square=True, cbar_kws={'shrink': 0.7})
plt.title('Matriz de correlación entre variables', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Mostrar correlación de cada feature con 'lesion' (ordenada de mayor a menor)
print('Correlación de cada variable con "lesion":')
corr_with_target = corr['lesion'].drop('lesion').sort_values(ascending=False)
print(corr_with_target.to_string())
```



Asignación de variables Feature y Target dentro de variables específicas

Paso 4: Seleccionar Variables

Variable objetivo (y): lesion — lo que queremos predecir
Variables predictoras (X): las 10 variables numéricas del pipeline

```
# Separar predictores (X) y objetivo (y)
X = df[feature_cols].copy()
y = df[lesion].copy()

# Verificar dimensiones y distribución
print('Variables seleccionadas:')
print(' Variables predictoras (X):', (list(X.columns)))
print(' Variable objetivo (y):', lesion)
print()
print(' Dimensiones de X:', (X.shape))
print(' Dimensiones de y:', (y.shape))
print(' Distribución de y:')
print(' (y.value_counts().to_string())')

Variables seleccionadas:
Variables predictoras (X): ['rpm', 'peso_levantado_kg', 'repeticiones_logradas', 'numero_serie', 'dificultad', 'porcentaje_estimulo', 'edad', 'peso_corporal_kg', 'duracion_minutos', 'cantidad_serie']
Variable objetivo (y): lesion

Dimensiones de X: (17879, 10)
Dimensiones de y: (17879,)
Distribución de y:
lesion
0    13786
1     5373
```

Paso 5: Dividir en Entrenamiento y Prueba

Proporción: **80% entrenamiento / 20% prueba**
Con `stratify=y` garantizamos que ambos conjuntos mantengan la misma proporción de lesiones que el dataset original.

```
# Dividir en train/test con estratificación para mantener balance de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

División de datos para pruebas y entrenamiento

Paso 5: Dividir en Entrenamiento y Prueba

Proporción: **80% entrenamiento / 20% prueba**
Con `stratify=y` garantizamos que ambos conjuntos mantengan la misma proporción de lesiones que el dataset original.

```
# Dividir en train/test con estratificación para mantener balance de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Verificar tamaño y balance de clases en cada conjunto
print('División completada:')
print(' Total de registros: ', (len(df)))
print(' Entrenamiento: ', (len(X_train)), registros ((len(X_train)/len(df)*100:.0f) '%'))
print(' Prueba: ', (len(X_test)), registros ((len(X_test)/len(df)*100:.0f) '%'))
print()
print(' Balance de clases (tasa de lesión) por conjunto:')
print(' Entrenamiento: ', (y_train.mean()*100) '%')
print(' Prueba: ', (y_test.mean()*100) '%')
```

```
División completada:
Total de registros: 17879
Entrenamiento: 13863 registros (80%)
Prueba: 3416 registros (20%)

Balance de Clases (tasa de lesión) por conjunto:
Entrenamiento: 31.46%
Prueba: 31.47%
```

5.1 Estandarización

Los modelos lineales (Regresión Logística) requieren que las variables estén en la misma escala. Aplicamos `StandardScaler`: media 0, desviación estándar 1.

```
# Estandarizar: centrar (media 0) y escalar (std 1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Ajusta y transforma train
X_test_scaled = scaler.transform(X_test) # Transforma test con los mismos parámetros

print('Estandarización aplicada (media=0, std=1):')
print(' X_train_scaled:', (X_train_scaled.shape))
print(' X_test_scaled:', (X_test_scaled.shape))
```

```
Estandarización aplicada (media=0, std=1):
X_train_scaled: (13863, 10)
X_test_scaled: (3416, 10)
```

Construcción y entrenamiento de modelo

Paso 6: Construir y Entrenar Modelos

En este paso construimos (configuramos) y entrenamos cada modelo. Primero exploramos tres variantes de Regresión Logística con diferentes tipos de regularización, y luego un Random Forest.

6.1 Regresión Logística — L1 (Lasso), L2 (Ridge) y ElasticNet

Variante	Regularización	Efecto
L1 (Lasso)	Penaliza suma de valores absolutos	Elimina variables poco importantes (selección de características)
L2 (Ridge)	Penaliza suma de cuadrados	Reduce coeficientes sin eliminarlos
ElasticNet	Combina L1 + L2	Balance entre selección y contracción

```
# Diccionario con las 3 variantes de Regresión Logística
modelos_lr = {
    'Logistic Regression (L1 - Lasso)': LogisticRegression(penalty='l1', solver='saga', C=1.0, max_iter=1000, random_state=42),
    'Logistic Regression (L2 - Ridge)': LogisticRegression(penalty='l2', solver='lbfgs', C=1.0, max_iter=1000, random_state=42),
    'Logistic Regression (ElasticNet)': LogisticRegression(penalty='elasticnet', solver='saga', C=1.0, l1_ratio=0.5, max_iter=1000, random_state=42),
}

resultados = [] # Lista para almacenar métricas de todos los modelos

# Entrenar y evaluar cada variante de regresión logística
for nombre, modelo in modelos_lr.items():
    modelo.fit(X_train_scaled, y_train)
    y_pred = modelo.predict(X_test_scaled)
    y_prob = modelo.predict_proba(X_test_scaled)[0, 1] # Probabilidad de clase positiva

    # Calcular métricas de rendimiento
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc_val = auc(fpr, tpr)
    gini = 2 * auc_val - 1 # Índice Gini a partir del AUC

    # Guardar resultados
    resultados.append({
        'modelo': nombre,
        'accuracy': acc,
        'precision': prec,
        'recall': rec,
        'f1': f1,
        'auc': auc_val,
        'gini': gini,
    })
```

Visualización de matriz de confusión en la regresión logística

6.2 Matriz de Confusión — Logistic Regression (L2)

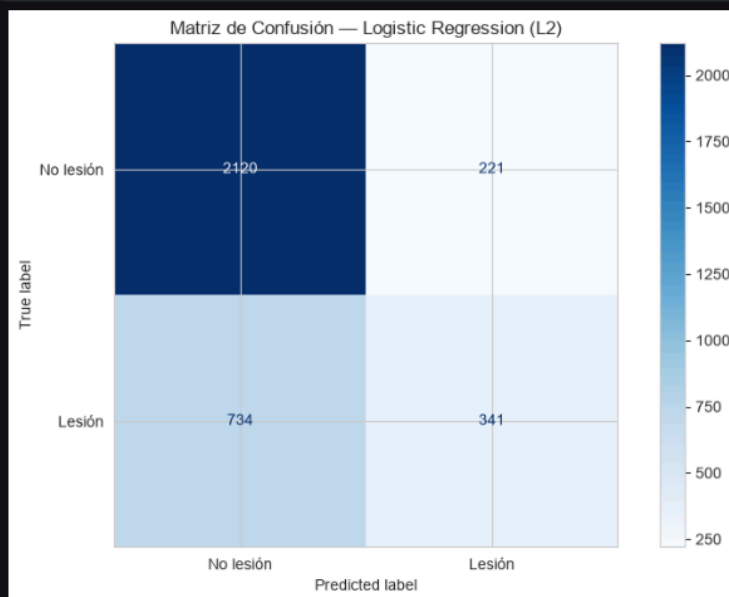
Tomamos el modelo con L2 (Ridge) como representante de la familia de regresión logística para visualizar sus aciertos y errores.

```
# Seleccionar el modelo L2 como representante de regresión logística
best_lr = modelos_lr['Logistic Regression (L2 - Ridge)']
y_pred_lr = best_lr.predict(X_test_scaled)

# Matriz de confusión y reporte de clasificación
cm = confusion_matrix(y_test, y_pred_lr)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No lesión', 'Lesión'])
disp.plot(cmap='Blues', values_format='d')
plt.title('Matriz de Confusión — Logistic Regression (L2)')
plt.show()

print('Reporte de clasificación:')
print(classification_report(y_test, y_pred_lr, target_names=['No lesión', 'Lesión']))
```

[12]



Reporte de clasificación:

Construcción del modelo Random Forest

6.3 Random Forest

Random Forest es un algoritmo de ensemble basado en múltiples árboles de decisión. Según la propuesta original, es el algoritmo principal de este proyecto. Usamos 150 árboles con profundidad máxima 12.

```
# Configurar y entrenar Random Forest con balance de clases
rf = RandomForestClassifier(n_estimators=150, max_depth=12, min_samples_leaf=5,
                           random_state=42, class_weight='balanced')
rf.fit(X_train, y_train) # NOTA: RF no necesita datos estandarizados

# Predicciones y probabilidades
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[0, 1]

# Calcular todas las métricas
acc_rf = accuracy_score(y_test, y_pred_rf)
prec_rf = precision_score(y_test, y_pred_rf)
rec_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
auc_rf = auc(fpr_rf, tpr_rf)
gini_rf = 2 * auc_rf - 1

# Guardar resultados
resultados.append({
    'modelo': 'Random Forest',
    'accuracy': acc_rf,
    'precision': prec_rf,
    'recall': rec_rf,
    'f1': f1_rf,
    'auc': auc_rf,
    'gini': gini_rf,
})

print('\n== Random Forest ==')
print(f'Accuracy : {acc_rf:.4f}')
print(f'Precision: {prec_rf:.4f}')
print(f'Recall   : {rec_rf:.4f}')
print(f'F1-score  : {f1_rf:.4f}')
print(f'AUC      : {auc_rf:.4f}')
print(f'Gini     : {gini_rf:.4f}')
```

```
== Random Forest ==
Accuracy : 0.6818
Precision: 0.4958
Recall   : 0.4959
F1-score : 0.3661
AUC      : 0.7351
Gini     : 0.4792
```

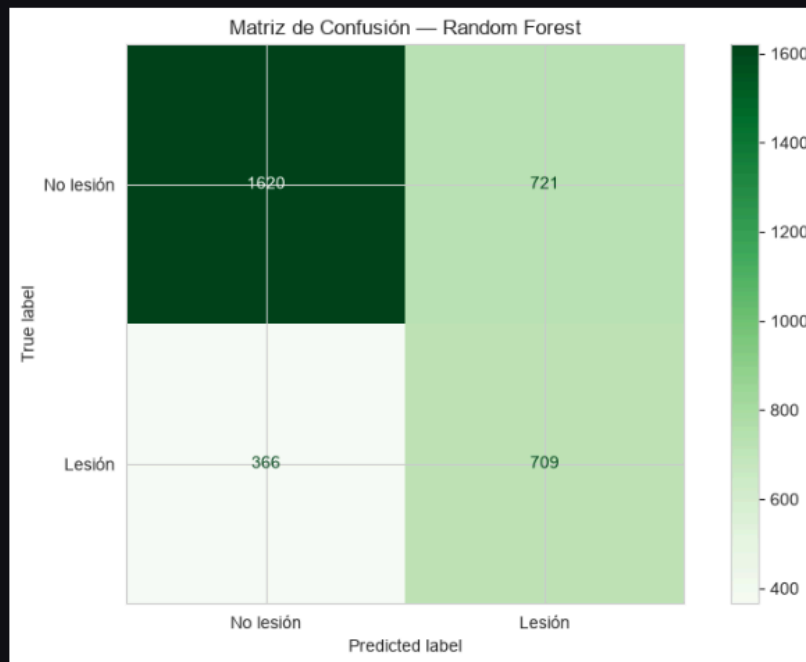
Matriz de confusión en Random Forest

Creacion de matriz de confusión para Random Forest

```
# Matriz de confusi\u00f3n para Random Forest
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf, display_labels=['No lesi\u00f3n', 'Lesi\u00f3n'])
disp_rf.plot(cmap='Greens', values_format='d')
plt.title('Matriz de Confusi\u00f3n - Random Forest')
plt.show()

print('Reporte de clasificaci\u00f3n:')
print(classification_report(y_test, y_pred_rf, target_names=['No lesi\u00f3n', 'Lesi\u00f3n']))
```

[14]



Reporte de clasificación:

	precision	recall	f1-score	support
No lesión	0.82	0.69	0.75	2341
Lesión	0.50	0.66	0.57	1075
accuracy			0.68	3416
macro avg	0.66	0.68	0.66	3416
weighted avg	0.72	0.68	0.69	3416

Visualización de variables a base de su importancia

Paso 7: Evaluar Modelos y Comparar Resultados

Usamos el conjunto de **prueba** (datos que ningún modelo ha visto) para medir el desempeño real de cada uno.

```
# Tabla comparativa de métricas para todos los modelos
df_resultados = pd.DataFrame(resultados).set_index('modelo')
df_resultados.round(4)
```

modelo	accuracy	precision	recall	f1	auc	gini
Logistic Regression (L1 - Lasso)	0.7198	0.6050	0.3163	0.4154	0.7406	0.4813
Logistic Regression (L2 - Ridge)	0.7204	0.6068	0.3172	0.4166	0.7407	0.4813
Logistic Regression (ElasticNet)	0.7204	0.6068	0.3172	0.4166	0.7406	0.4813
Random Forest	0.6818	0.4958	0.5995	0.5661	0.7351	0.4702

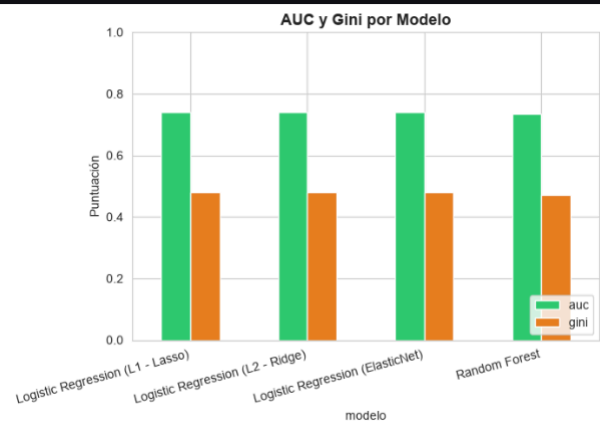
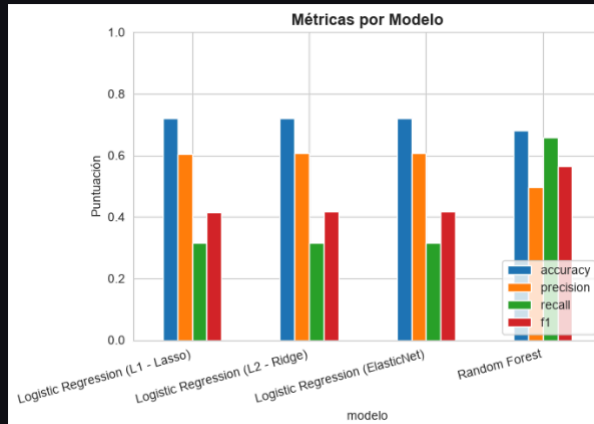
Matriz de comparación

```
# Gráfico comparativo de métricas entre modelos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Primer gráfico: accuracy, precision, recall, f1
df_resultados[['accuracy', 'precision', 'recall', 'f1']].plot(kind='bar', ax=axes[0])
axes[0].set_title('Métricas por Modelo', fontsize=13, fontweight='bold')
axes[0].set_xlabel(0, 3)
axes[0].set_ylabel('Puntuación')
axes[0].legend(loc='lower right')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=15, ha='right')

# Segundo gráfico: AUC y Gini
df_resultados[['auc', 'gini']].plot(kind='bar', ax=axes[1], color=[ '#2ecc71', '#e67e22' ])
axes[1].set_title('AUC y Gini por Modelo', fontsize=13, fontweight='bold')
axes[1].set_xlabel(0, 3)
axes[1].set_ylabel('Puntuación')
axes[1].legend(loc='lower right')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=15, ha='right')

plt.tight_layout()
plt.show()
```



Comparación de modelos con ROC

Paso 8: Comparación de Modelos con Curva ROC

La curva ROC muestra visualmente la capacidad de cada modelo para separar las clases. Mientras más cerca esté la curva de la esquina superior izquierda, mejor.

```
# Curva ROC comparativa entre modelos seleccionados
plt.figure(figsize=(8, 6))

# Preparar modelos y datos para la curva ROC
modelos_roc = [
    ('Logistic Regression (L2)', modelos_lr['Logistic Regression (L2 - Ridge)', X_test_scaled],
    ('Logistic Regression (L1)', modelos_lr['Logistic Regression (L1 - Lasso)', X_test_scaled],
    ('Random Forest', rf, X_test),
]

for nombre, model, X_vals in modelos_roc:
    y_prob = model.predict_proba(X_vals)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc_val = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{nombre} (AUC = {auc_val:.3f})', linewidth=2)

# Línea diagonal correspondiente a un clasificador aleatorio
plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio (AUC = 0.5)', alpha=0.7)
plt.xlabel('Tasa de Falsos Positivos (1 - Especificidad)', fontsize=11)
plt.ylabel('Tasa de Verdaderos Positivos (Recall)', fontsize=11)
plt.title('Curva ROC - Comparación de Modelos', fontsize=14, fontweight='bold')
plt.legend(loc='lower right', fontsize=10)
plt.grid(alpha=0.3)
plt.tight_layout()
```

